CO1 PRACTICAL EXERCISE

1. **Install Matplotlib and display a simple line graph.**

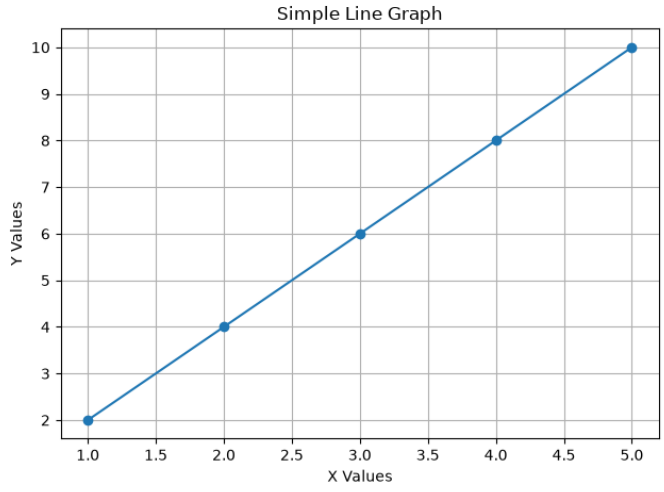
CODE:

```
import matplotlib.pyplot as plt


x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]


plt.plot(x, y, marker="o")
plt.title("Simple Line Graph")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.grid(True)
plt.tight_layout()
plt.show()
```

OUTPUT:



2. Install Scikit-learn and print its version.

CODE:

```
import sklearn


print("Scikit-learn version:", sklearn.__version__)
```

OUTPUT:

```
PS C:\Users\janur\Downloads\GEN AI> python "C:\Users\janur\Downloads\GEN AI\PRACTICAL EXERCISES\experiment2.py"
Scikit-learn installed successfully!
Scikit-learn version: 1.9.0
```

3. Create a NumPy array containing the first 20 natural numbers.

CODE:

```
import numpy as np

numbers = np.arange(1, 21)

print("First 20 natural numbers:")

print(numbers)
```

OUTPUT:

```
First 20 natural numbers:
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
```

4. Create a Pandas DataFrame with student names and grades.

CODE:

```
import pandas as pd


data = {
    "Student Name": ["Arun", "Priya", "Rahul", "Divya"],
    "Grade": ["A", "B", "A+", "B+"],
}
df = pd.DataFrame(data)
print(df.to_string(index=False))
```

OUTPUT:

```
  Student Name Grade
0         Arun     A
1        Priya     B
2        Rahul    A+
3        Divya    B+
```

5. Verify the installed versions of all AI libraries on your system.

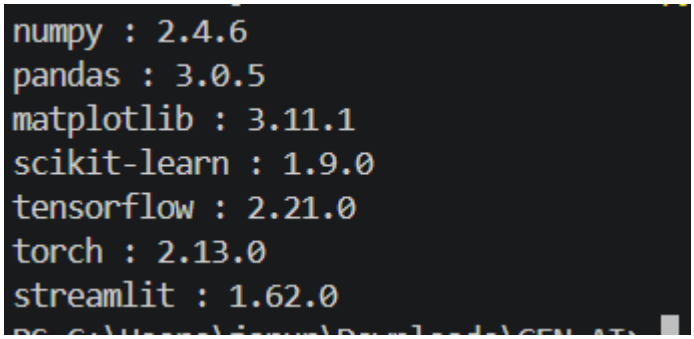
CODE:

```
from importlib.metadata import version, PackageNotFoundError


packages = [
    "numpy", "pandas", "matplotlib", "scikit-learn",
    "tensorflow", "torch", "transformers", "streamlit"
]


for package in packages:
    try:
        print(f"{package}: {version(package)}")
    except PackageNotFoundError:
        print(f"{package}: Not installed")
```

OUTPUT:



6. Create tensors using PyTorch and perform basic mathematical operations.

CODE:

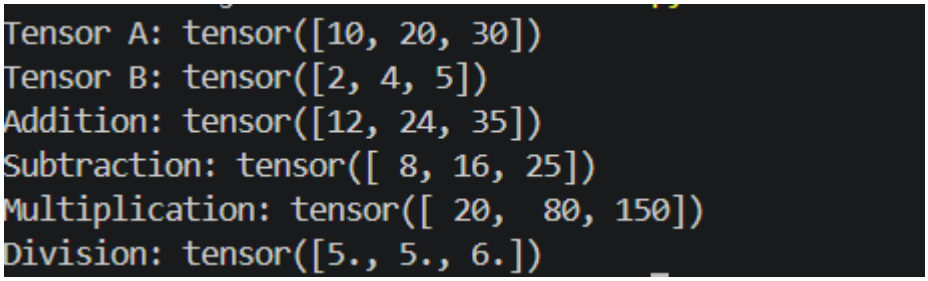
```
import torch


a = torch.tensor([10, 20, 30])
b = torch.tensor([2, 4, 5])


print("Tensor A:", a)
```

```
print("Tensor B:", b)
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
```

OUTPUT:



7. Load a Pre-trained Transformer Model using Hugging Face and perform sentiment analysis.

CODE:

```
from transformers import pipeline

classifier = pipeline(
    "sentiment-analysis",
    model="distilbert/distilbert-base-uncased-finetuned-sst-2-english"
)

sentence = input("Enter a sentence for sentiment analysis: ").strip()
if not sentence:
    sentence = "I really enjoyed this movie."

result = classifier(sentence)[0]
print("Input:", sentence)
print("Sentiment:", result["label"])
print("Confidence:", round(float(result["score"]), 4))
```

OUTPUT:

```
Input: I really enjoyed this movie.
Sentiment: POSITIVE
Confidence: 0.9999
```

8. Tokenization using BERT Tokenizer

CODE:

```
from transformers import AutoTokenizer


tokenizer = AutoTokenizer.from_pretrained("google-bert/bert-base-uncased")
sentence = input("Enter a sentence: ").strip() or "Artificial intelligence is useful."


print("Sentence:", sentence)
print("BERT tokens:", tokenizer.tokenize(sentence))
```

OUTPUT:

```
Enter a sentence: ai is very useful
Sentence: ai is very useful
BERT tokens: ['ai', 'is', 'very', 'useful']
```

9. Generate text using a pre-trained GPT-2 model.

CODE:

```
from transformers import pipeline


generator = pipeline("text-generation", model="openai-community/gpt2")
prompt = input("Enter a prompt: ").strip() or "Artificial intelligence can help"


result = generator(
    prompt,
    max_new_tokens=30,
    do_sample=False,
    pad_token_id=generator.tokenizer.eos_token_id
)
print("Generated text:")
```

```
print(result[0]["generated_text"])
```

OUTPUT:

```
Generated text:
ai can help you get the most out of your life.

The best way to get the most out of your life is to be a good person.
```

10. Load a pre-trained transformer model using the Hugging Face pipeline() API and perform sentiment analysis.

CODE:

```
from transformers import pipeline


sentiment = pipeline(
    "sentiment-analysis",
    model="distilbert/distilbert-base-uncased-finetuned-sst-2-english"
)
review = input("Enter a review: ").strip() or "The product quality is excellent."
result = sentiment(review)[0]


print("Review:", review)
print("Label:", result["label"])
print("Score:", round(float(result["score"]), 4))
```

OUTPUT:

```
Enter a review: The product quality is excellent
Review: The product quality is excellent
Label: POSITIVE
Score: 0.9999
```

11. Load the BERT tokenizer and tokenize a sentence.

CODE:

```
from transformers import AutoTokenizer


tokenizer = AutoTokenizer.from_pretrained("google-bert/bert-base-uncased")
sentence = input("Enter a sentence: ").strip() or "I am learning Generative AI."
```

```
print("Original sentence:", sentence)
print("Tokenized sentence:", tokenizer.tokenize(sentence))
```

OUTPUT:

```
Enter a sentence: i am learning coding
Original sentence: i am learning coding
Tokenized sentence: ['i', 'am', 'learning', 'coding']
```

12. Load a pre-trained BERT model and generate contextual embeddings.

CODE:

```
import torch
from transformers import AutoTokenizer, AutoModel

model_name = "google-bert/bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)
sentence = input("Enter a sentence: ").strip() or "AI helps people solve problems."

inputs = tokenizer(sentence, return_tensors="pt")
model.eval()
with torch.no_grad():
    outputs = model(**inputs)

embeddings = outputs.last_hidden_state
print("Input:", sentence)
print("Embedding tensor shape (batch, tokens, dimensions):", tuple(embeddings.shape))
print("First token, first 5 embedding values:", embeddings[0, 0, :5].tolist())
```

OUTPUT:

```
Enter a sentence: ai helps people solve problems
Input: ai helps people solve problems
Embedding tensor shape (batch, tokens, dimensions): (1, 7, 768)
First token, first 5 embedding values: [-0.004043924622237682, 0.059553660452365875, -0.17591559886932373, 0.17468194663524628
, -0.3566426932811737]
```

13. Generate text using a pre-trained GPT-2 model.

CODE:

```
from transformers import pipeline


generator = pipeline("text-generation", model="openai-community/gpt2")
prompt = input("Enter a prompt: ").strip() or "The future of technology"


result = generator(
    prompt,
    max_new_tokens=25,
    do_sample=False,
    pad_token_id=generator.tokenizer.eos_token_id
)
print("Prompt:", prompt)
print("Generated text:", result[0]["generated_text"])
```

OUTPUT:

```
Prompt: the future of technology
Generated text: the future of technology, and the future of the world.

The future of technology, and the future of the world.

The
```

14. Question Answering using a Pre-trained Transformer

CODE:

```
import os
os.environ["USE_TF"] = "0"


from transformers import pipeline


qa = pipeline(
    "question-answering",
```

```
    model="distilbert/distilbert-base-cased-distilled-squad",
    framework="pt"
)

context = "Python is a programming language used in artificial intelligence."
question = "Which programming language is used in artificial intelligence?"

result = qa(question=question, context=context)

print("Context:", context)
print("Question:", question)
print("Answer:", result["answer"])
print("Confidence:", round(result["score"], 4))
```

OUTPUT:

```
Context: Python is a programming language used in artificial intelligence.
Question: Which programming language is used in artificial intelligence?
Answer: Python
Confidence: 0.9992
```

15. Tokenize a sentence using the BERT tokenizer and display the generated tokens and token IDs.

CODE:

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("google-bert/bert-base-uncased")
sentence = input("Enter a sentence: ").strip() or "I love artificial intelligence."

tokens = tokenizer.tokenize(sentence)
token_ids = tokenizer.convert_tokens_to_ids(tokens)

print("Sentence:", sentence)
print("Tokens:", tokens)
```

```
print("Token IDs:", token_ids)
```

OUTPUT:

```
Enter a sentence: i like artificial intelligence
Sentence: i like artificial intelligence
Tokens: ['i', 'like', 'artificial', 'intelligence']
Token IDs: [1045, 2066, 7976, 4454]
```

16. Compare the tokenization outputs of BERT and GPT-2.

CODE:

```
from transformers import AutoTokenizer


bert = AutoTokenizer.from_pretrained("google-bert/bert-base-uncased")
gpt2 = AutoTokenizer.from_pretrained("openai-community/gpt2")
sentence = input("Enter a sentence: ").strip() or "I love Generative AI!"


print("Sentence:", sentence)
print("BERT tokens:", bert.tokenize(sentence))
print("GPT-2 tokens:", gpt2.tokenize(sentence))
```

OUTPUT:

```
Enter a sentence: i love gen ai
Sentence: i love gen ai
BERT tokens: ['i', 'love', 'gen', 'ai']
GPT-2 tokens: ['i', 'Ġlove', 'Ġgen', 'Ġa', 'i']
```

17. Generate contextual embeddings for a sentence using BERT.

CODE:

```
import torch
from transformers import AutoTokenizer, AutoModel


model_name = "google-bert/bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)
model.eval()
```

```
sentence = input("Enter a sentence: ").strip() or "Machine learning is powerful."
inputs = tokenizer(sentence, return_tensors="pt")

with torch.no_grad():
    outputs = model(**inputs)

tokens = tokenizer.convert_ids_to_tokens(inputs["input_ids"][0])
embeddings = outputs.last_hidden_state[0]

for token, vector in zip(tokens, embeddings):
    print(f"{token}: shape={tuple(vector.shape)}, first 3 values={vector[:3].tolist()}")
```

OUTPUT:

```
Enter a sentence: machine learning is powerful
[CLS]: shape=(768,), first 3 values=[-0.18669536709785461, 0.14583030343055725, -0.1653253585100174]
machine: shape=(768,), first 3 values=[0.04004612937569618, 0.16689473390579224, -0.3493814766407013]
learning: shape=(768,), first 3 values=[-0.5255810022354126, 0.12258405238389969, -0.12636035680770874]
is: shape=(768,), first 3 values=[-0.5107489824295044, -0.13650193810462952, 0.19334439933300018]
powerful: shape=(768,), first 3 values=[-0.42273783683776855, -0.7385610342025757, 0.11657892167568207]
[SEP]: shape=(768,), first 3 values=[0.8371703624725342, 0.026699528098106384, -0.3807775676250458]
```

18. Compare contextual embeddings generated for two semantically similar sentences.

CODE:

```
import torch
import torch.nn.functional as F
from transformers import AutoTokenizer, AutoModel

model_name = "google-bert/bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)
model.eval()

sentence1 = input("Enter sentence 1: ").strip() or "The cat is sitting on the mat."
```

```
sentence2 = input("Enter sentence 2: ").strip() or "A cat is resting on a rug."

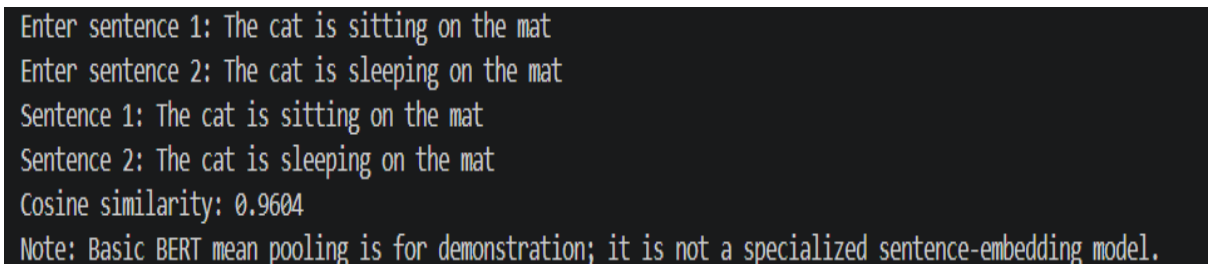

inputs = tokenizer(
    [sentence1, sentence2],
    padding=True,
    truncation=True,
    return_tensors="pt"
)


with torch.no_grad():
    token_embeddings = model(**inputs).last_hidden_state


mask = inputs["attention_mask"].unsqueeze(-1).to(token_embeddings.dtype)
sentence_embeddings = (token_embeddings * mask).sum(dim=1) /
mask.sum(dim=1).clamp(min=1)
similarity = F.cosine_similarity(
    sentence_embeddings[0].unsqueeze(0),
    sentence_embeddings[1].unsqueeze(0)
).item()


print("Sentence 1:", sentence1)
print("Sentence 2:", sentence2)
print("Cosine similarity:", round(similarity, 4))
print("Note: Basic BERT mean pooling is for demonstration; it is not a specialized sentence-
embedding model.")
```

OUTPUT:



19. Generate text from a prompt using the GPT-2 language model.

CODE:

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
model_name = "openai-community/gpt2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)
model.eval()
prompt = input("Enter a prompt: ")
inputs = tokenizer(prompt, return_tensors="pt")
with torch.no_grad():
    output = model.generate(
        **inputs,
        max_new_tokens=40,
        do_sample=False,
        pad_token_id=tokenizer.eos_token_id
    )
generated_text = tokenizer.decode(
    output[0],
    skip_special_tokens=True
)
print("\nGenerated text:")
print(generated_text)
```

OUTPUT:

```
Enter a prompt: AI IS GOOD SUBJECT

Generated text:
AI IS GOOD SUBJECT: The Islamic State of Iraq and Syria (ISIS) is a terrorist organization that has been operating in Iraq sin
ce 2011. It is a terrorist organization that has been operating in Iraq since 2011. It is
```

20. Generate and compare responses for multiple prompts using a pre-trained LLM.

CODE:

```
from transformers import pipeline


generator = pipeline("text-generation", model="openai-community/gpt2")
prompts = [
    "Artificial intelligence is",
    "Climate change can be reduced by",
    "The future of education"
]


for prompt in prompts:
    result = generator(
        prompt,
        max_new_tokens=25,
        do_sample=False,
        pad_token_id=generator.tokenizer.eos_token_id
    )
    print("\nPrompt:", prompt)
    print("Generated response:", result[0]["generated_text"])
```

OUTPUT:

```
Prompt: Artificial intelligence is
Generated response: Artificial intelligence is a new field of research that has been in the works for a while now. It is a fie
ld that has been in the
[transformers] Both `max_new_tokens` (=25) and `max_length`(=50) seem to have been set. `max_new_tokens` will take precedence.
 Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_g
eneration)

Prompt: Climate change can be reduced by
Generated response: Climate change can be reduced by reducing the amount of carbon dioxide in the atmosphere.

The study, published in the journal Nature Climate Change, found that
[transformers] Both `max_new_tokens` (=25) and `max_length`(=50) seem to have been set. `max_new_tokens` will take precedence.
 Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_g
eneration)

Prompt: The future of education
Generated response: The future of education is not about the future of the classroom. It's about the future of the classroom.

The future of education is
```